

CS224n & Transformer

2026.05.07

Hayeong Jung

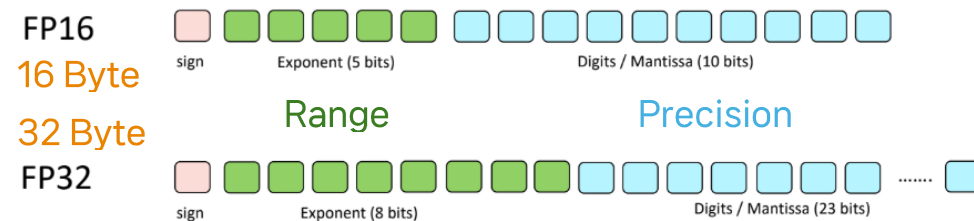
Contents

- **CS224n**
 - Lecture 12: Efficient Training
 - Lecture 13: Brain-Computer Interfaces
- **Code Review**
 - Annotated Transformer
 - Train & Test

Lecture 12: Efficient Training

• Mixed Precision Training

• Floating Points



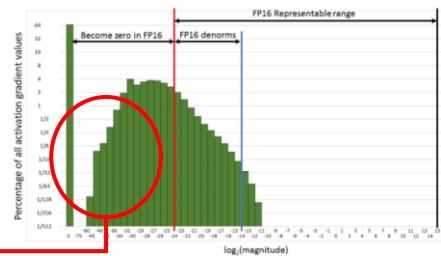
• FP32 → FP16

• Pros

- Low Memory Usage
- Solution of OOM

• Cons

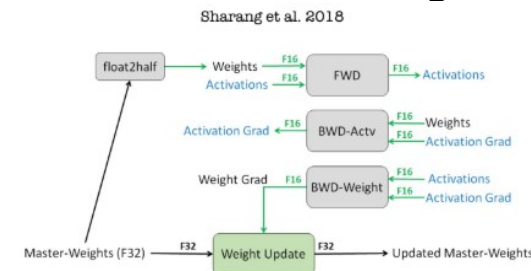
- Cause Underflow
- Imprecise Update



Half of Gradients go to Zero
→ Underflow!

• Mixed Precision Training

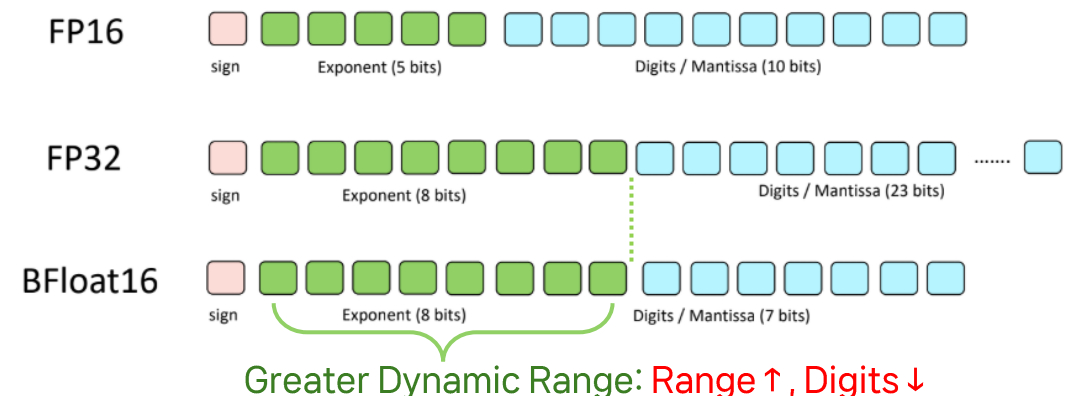
- Idea
- Gradient Scaling



Recipe for Mixed-Precision Training

1. Maintain a copy of model parameters in FP32 (Master weights)
2. Run forward pass in FP16
3. Scale loss by a large value (to artificially increase gradient)
4. Compute gradient in FP16
5. Copy gradient into FP32 and divide by scale factor
6. Update master weights in FP32 [fixes weight update issue!]
7. Copy into FP16 version

• BFloat16

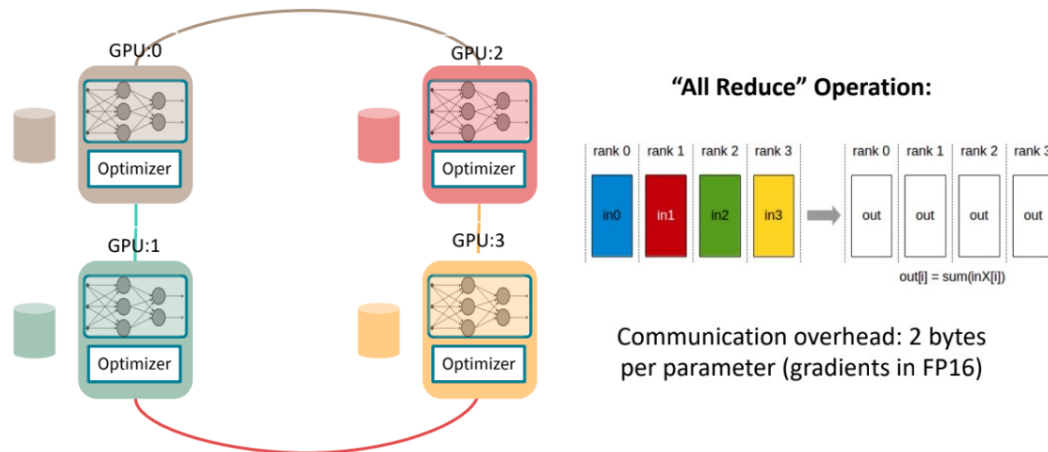


Lecture 12

- Multi-GPU Training

- (Naïve) DDP

- *Distributed Data Parallel*
 - All Reduce Operation;
2 bytes Overhead per Parameter
 - Poor Memory Scaling



- ZeRO

- Stage-1

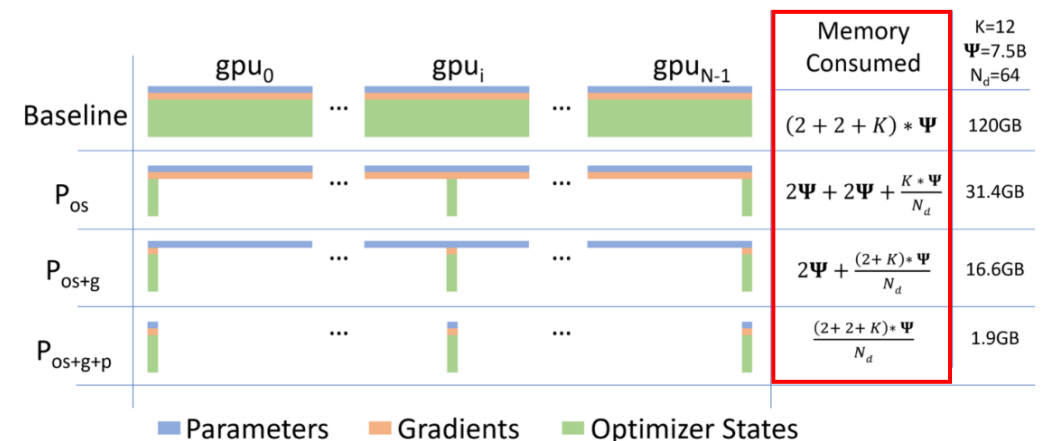
- Optimizer State Partitioning (P_{os})
 - Reduce Scatter Operation

- Stage-2

- Add Gradient Partitioning (P_{os+g})

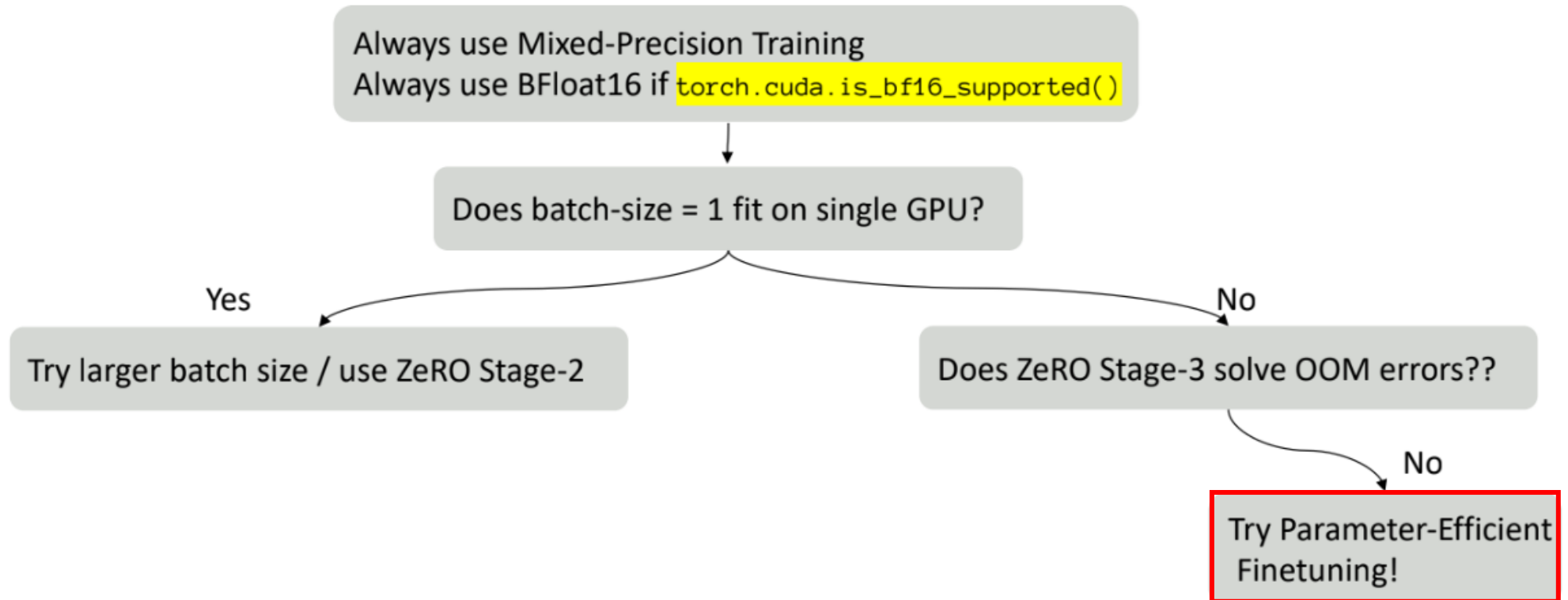
- Stage-3 (Full FSDP)

- Add Parameter Partitioning (P_{os+g+p})



Lecture 12

- Summary of Multi-GPU Training



Lecture 12

- Parameter Efficient Finetuning (PEFT)

- Necessity

- Models become Too Large
 - Full Finetuning is Impractical
 - Similar Performance to Full Finetuning

- Full Finetuning

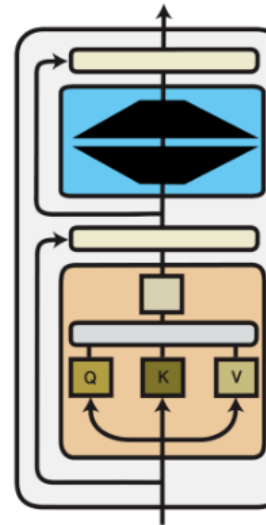
- Goal

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \log P_{\Phi}(y|x)$$

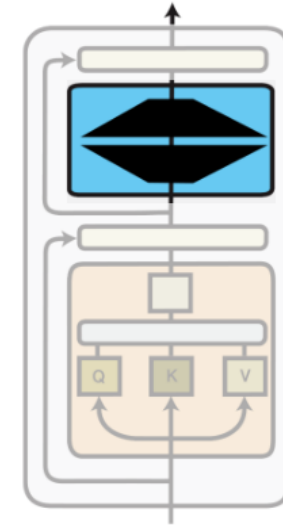
- Fine-tuned Parameter

$$\Phi = \Phi_0 + \Delta\Phi$$

⇒ Too Large! (GPT-3 has 175B Param)



Full Fine-tuning
Update **all model**
parameters

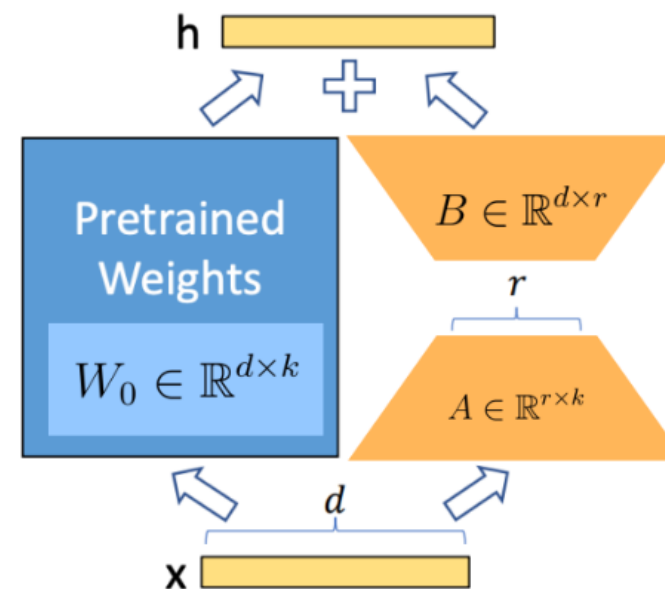


Parameter-efficient Fine-tuning
Update a **small subset** of model
parameters

Lecture 12

- LoRA
 - Weight Matrix Decomposition
 - $W_0 + \Delta W = W_0 + \alpha BA$
 -
 - $\alpha = \text{Tradeoff Rate}$
 - Advantages
 - Much Cheaper
 - No Additional Inference Latency
 - Implementation

```
def regular_forward_matmul(x, W):  
    h = x @ W  
    return h  
  
def lora_forward_matmul(x, W, W_A, W_B):  
    h = x @ W # regular matrix multiplication  
    h += x @ (W_A @ W_B)*alpha # use scaled LoRA weights  
    return h
```



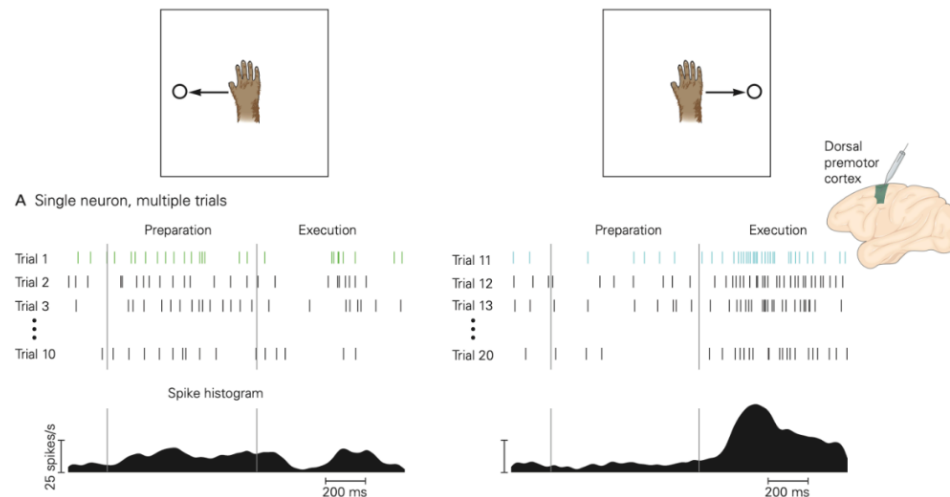
Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Lecture 13: Brain-Computer Interfaces

- **History of BCI**

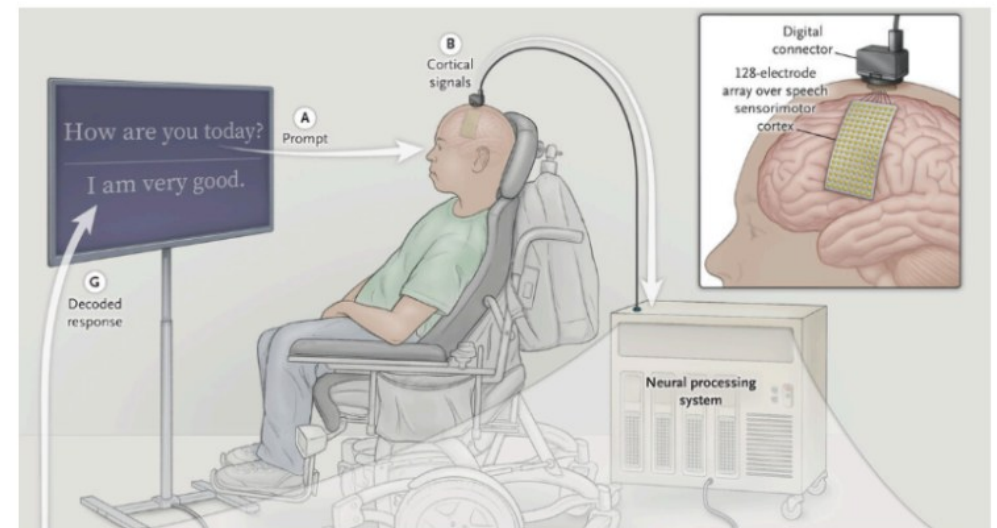
- Electric Signal in Brain
- Electroencephalogram(EEG)
 - Low Precision & Resolution
- Decode Signal Directly
 - Signal from Neurons in Motor Cortex

Different in Each Trials;
Noisy!



- **Language Processing in Brain**

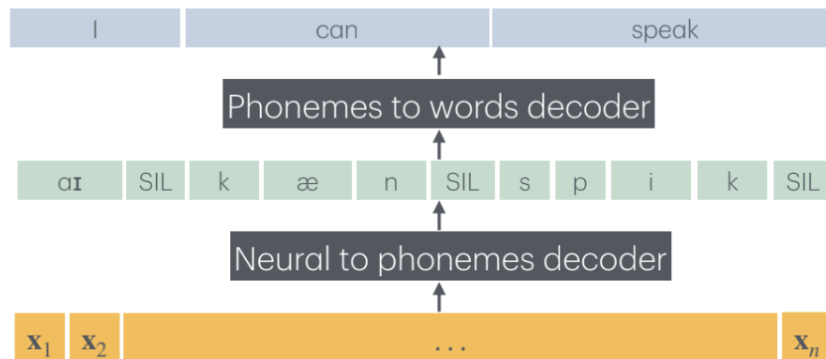
- Extremely Complex
- Produce Speech Sound
 - Motor Cortex
 - Small Vocabulary Speech : ECoG



Lecture 13

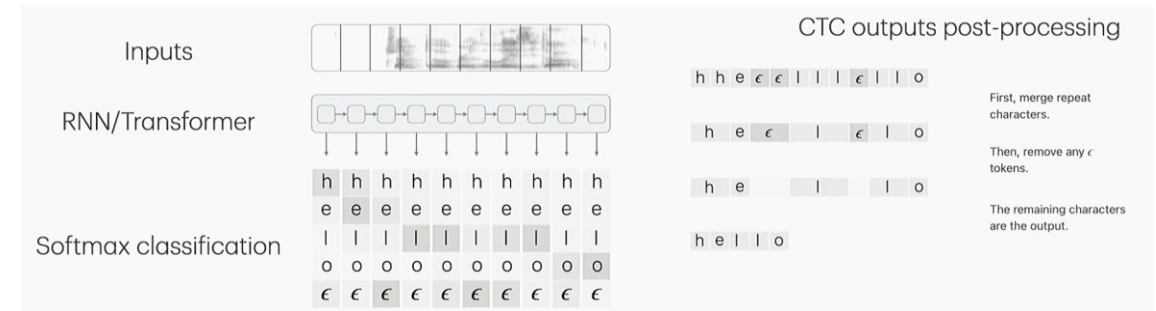
- High Performance Speech Neuroprosthesis

- Data Collection
 - ~ Selected from Switchboard Corpus
- Problem Definition



- Neural to Phonemes Decoder
 - Seq2Seq Problem
 - Monotonic Alignment

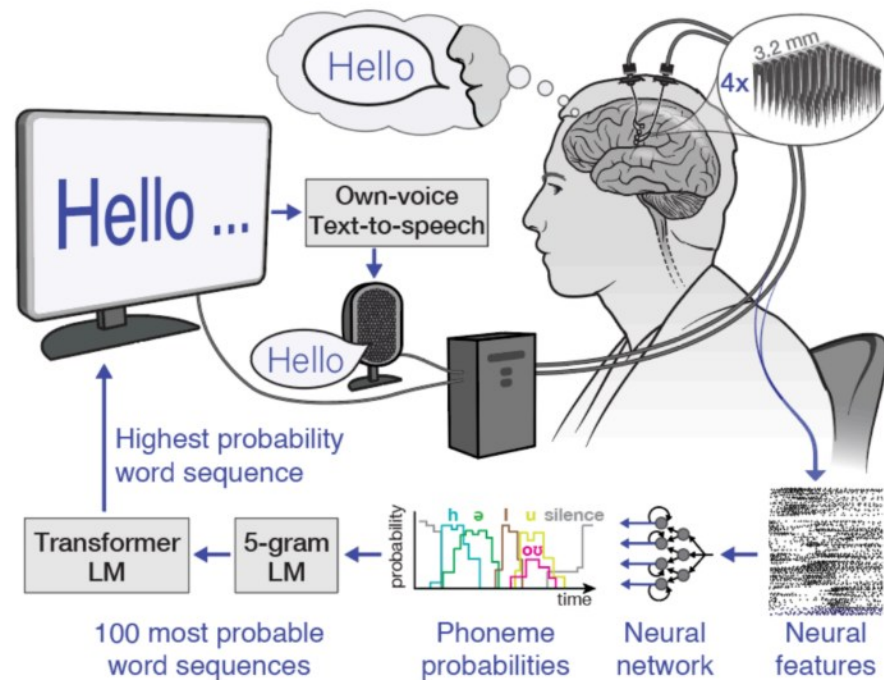
- Connectionist Temporal Classification (CTC)



- Proper Network
 - Transformer $\rightarrow X$
 - RNN $\rightarrow O$ (Especially, GRU)
 - Small Dataset
 - Short Dependency
 - Real-time Processing

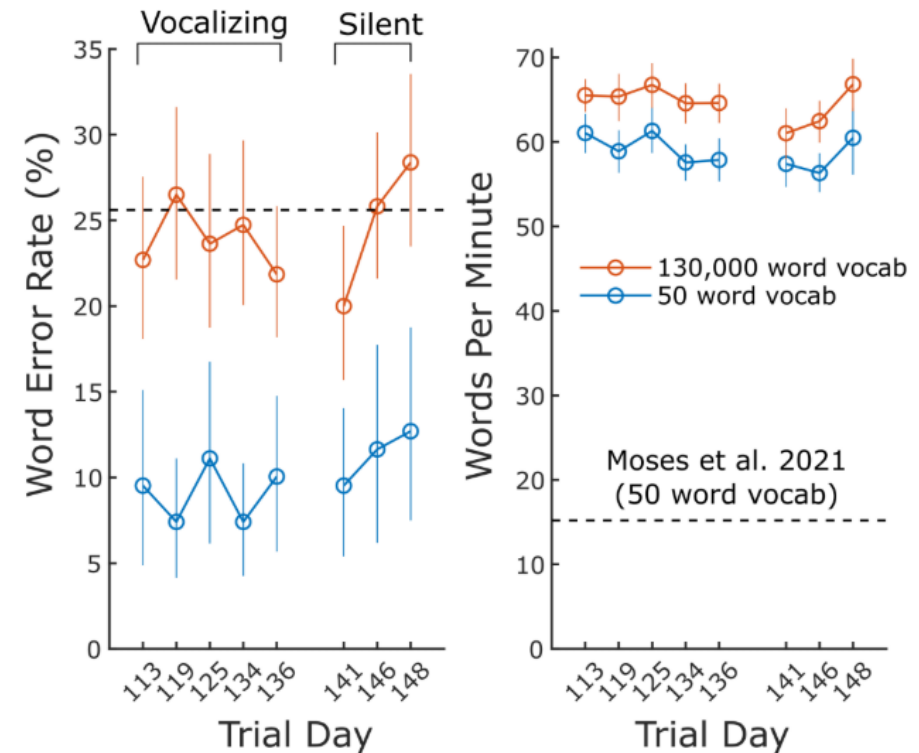
Lecture 13

- High Performance Speech Neuroprosthesis
 - Integrated Model

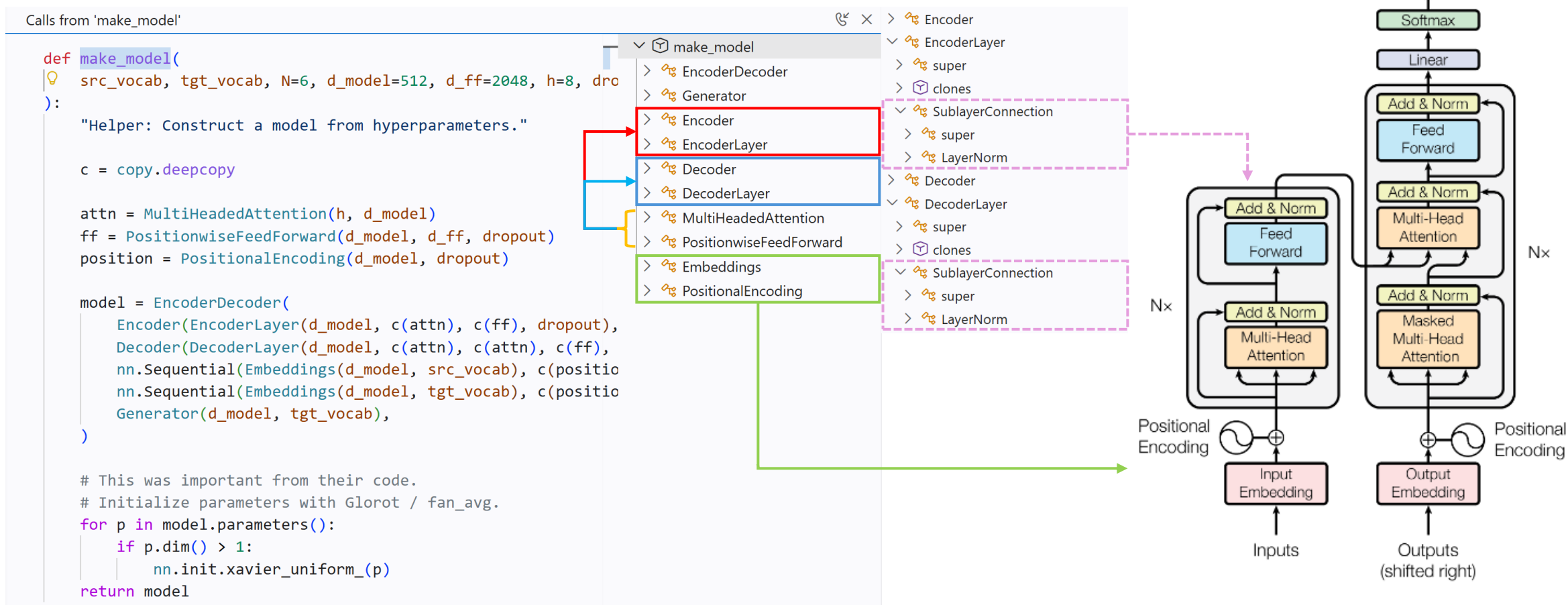


- Evaluation

- Word Error Rate $WER(\mathbf{Y}, \hat{\mathbf{Y}}) = \frac{\text{distance}(\mathbf{Y}, \hat{\mathbf{Y}})}{\text{length}(\mathbf{Y})}$



- **Transformer Architecture**



Code Review : Call Hierarchy

- **def** make_model()
 - **class** EncoderDecoder()
 - **class** Generator()
 - **class** Encoder()
 - **class** EncoderLayer()
 - **class** SublayerConnection()
 - **class** MultiHeadedAttention()
 - **class** PositionwiseFeedForward()
 - **class** LayerNorm()
 - **class** Decoder()
 - **class** DecoderLayer()
 - **class** SublayerConnection()
 - **class** MultiHeadedAttention()
 - **class** PositionwiseFeedForward()
 - **class** LayerNorm()
 - **class** Embeddings()
 - **class** PositionalEncoding()

Code Review: Model Architecture

- Encoder-Decoder ◀
- Encoder
- Decoder
- Attention
- Position-wise FFN
- Embeddings
- Positional Encoding

```
class EncoderDecoder(nn.Module):
    """
    A standard Encoder-Decoder architecture. Base for this and many
    other models.

    표준 Encoder-Decoder 구조. Transformer를 비롯한 많은 다른 모델의 바탕이 됨.
    """

    def __init__(self, encoder, decoder, src_embed, tgt_embed, generator):
        super(EncoderDecoder, self).__init__() # 부모 객체인 nn.Module도 함께 초기화

        # Encoder와 Decoder Structure를 받음
        self.encoder = encoder
        self.decoder = decoder

        # 입력과 출력 단어 임베딩 레이어를 받음
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed

        # 생성기(Generator)를 받음 (여기에서는 선형 변환 + SoftMax 변환을 사용 -> 확률 분포로의 변환)
        self.generator = generator

    def forward(self, src, tgt, src_mask, tgt_mask):
        "Take in and process masked src and target sequences."

        # 모델 호출 시 실행되는 메인 함수
        return self.decode(self.encode(src, src_mask), src_mask, tgt, tgt_mask)

    def encode(self, src, src_mask):
        return self.encoder(self.src_embed(src), src_mask) # src_mask는 <pad> 등 가리기 위해 사용

    def decode(self, memory, src_mask, tgt, tgt_mask):
        return self.decoder(self.tgt_embed(tgt), memory, src_mask, tgt_mask) # tgt_mask는 <pad> 및 미래 token 가리기 위해 사용
```

Code Review: Model Architecture

- Encoder-Decoder
- Encoder ◀
 - Encoder Layer
 - Sublayer Connection
 - Layer Normalization
- Decoder
- Attention
- Position-wise FFN
- Embeddings
- Positional Encoding

```
class Encoder(nn.Module):
    "Core encoder is a stack of N layers"

    # 하나의 EncoderLayer 객체를 입력받아 전체 Encoder를 (stack 형태로) 구성
    def __init__(self, layer, N):
        super(Encoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = LayerNorm(layer.size) # Layer Normalization, 아래에서 class로 정의

    def forward(self, x, mask):
        "Pass the input (and mask) through each layer in turn."
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

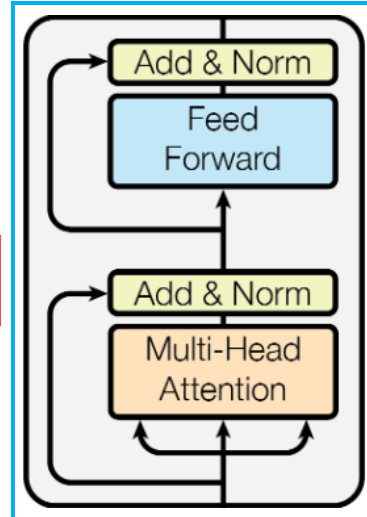
```
class EncoderLayer(nn.Module):
    "Encoder is made up of self-attn and feed forward (defined below)"

    def __init__(self, size, self_attn, feed_forward, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = self_attn # attention 및 ffn은 아래에 정의됨
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 2)
        self.size = size

    def forward(self, x, mask):
        "Follow Figure 1 (left) for connections."

        # self-attention sub-layer,
        # self_attn은 총 4개의 인자를 받아야 하기 때문에 lambda를 이용하여 norm_x가 3번 들어갈 수 있도록 함
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, mask))
        return self.sublayer[1](x, self.feed_forward) # feed forward sub-layer
```

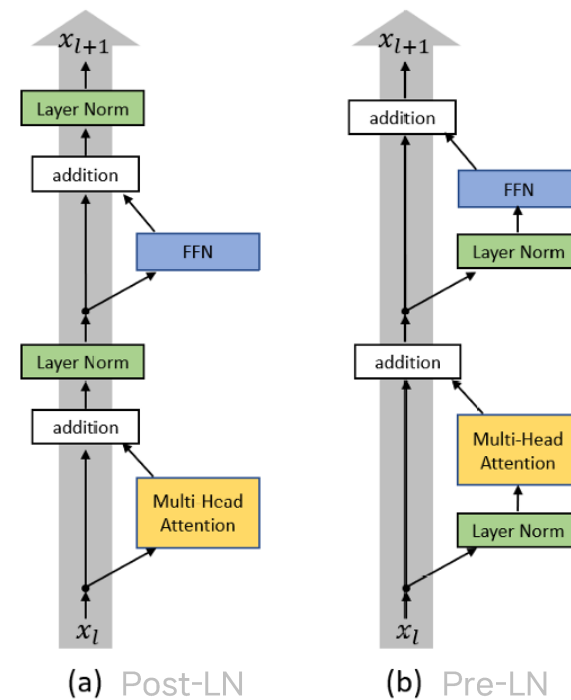
N×



Code Review: Model Architecture

- Encoder-Decoder
- Encoder ◀
 - Encoder Layer
 - Sublayer Connection
 - Layer Normalization
- ⇒ Post-LN vs Pre-LN
- Decoder
- Attention
- Position-wise FFN
- Embeddings
- Positional Encoding

```
class SublayerConnection(nn.Module):  
    """  
    A residual connection followed by a layer norm.  
    Note for code simplicity the norm is first as opposed to last.  
  
    (본래 논문에서는) Residual Connection이 먼저, 그 이후 Layer Normalization이 이루어진다.  
    다만 아래에서는, 코드의 간결함을 위하여 Normalization이 우선적으로 수행됨에 유의하라.  
    """  
  
    def __init__(self, size, dropout):  
        super(SublayerConnection, self).__init__()  
        self.norm = LayerNorm(size) # size == feature size  
        self.dropout = nn.Dropout(dropout)  
  
    def forward(self, x, sublayer): #sublayer로 self_attn or feed_forward를 입력받음  
        "Apply residual connection to any sublayer with the same size."  
        # 논문에서는: output = LayerNorm(x + Dropout(Sublayer(x))) Post-LN 구조지만,  
        # 코드에서는: output = x + Dropout(Sublayer(LayerNorm(x))) Pre-LN 구조를 사용하고 있음  
        return x + self.dropout(sublayer(self.norm(x)))  
  
class LayerNorm(nn.Module):  
    "Construct a layernorm module (See citation for details)."  
  
    def __init__(self, features, eps=1e-6):  
        super(LayerNorm, self).__init__()  
        self.a_2 = nn.Parameter(torch.ones(features)) # Scale, 학습 가능 파라미터, size는 feature와 동일  
        self.b_2 = nn.Parameter(torch.zeros(features)) # Bias, 학습 가능 파라미터  
        self.eps = eps # epsilon, 작은 양수값, std -> 0 일 때, 나누는 값이 0이 되지 않도록 더해주는 용도  
  
    def forward(self, x):  
        # 지정된 차원이 -1이므로 x의 shape가 [batch_size, seq_len, d_model]일 때,  
        # 마지막 차원인 d_model을 기준으로 평균 계산  
        mean = x.mean(-1, keepdim=True)  
        # keepdim은 'x - mean' 등의 연산에서 broadcasting을 용이하게 하기 위하여 마지막 차원 형태 유지  
        std = x.std(-1, keepdim=True)  
        return self.a_2 * (x - mean) / (std + self.eps) + self.b_2
```



On Layer Normalization in the Transformer Architecture(2020)

Code Review: Model Architecture

- Encoder-Decoder
- Encoder
- Decoder ◀
 - Decoder Layer
 - Self-attention
 - Source-attention
- Attention
- Position-wise FFN
- Embeddings
- Positional Encoding

```
class Decoder(nn.Module):  
    "Generic N layer decoder with masking."
```

```
    def __init__(self, layer, N):  
        super(Decoder, self).__init__()  
        self.layers = clones(layer, N)  
        self.norm = LayerNorm(layer.size)
```

```
    def forward(self, x, memory, src_mask, tgt_mask):  
        for layer in self.layers:  
            x = layer(x, memory, src_mask, tgt_mask)  
        return self.norm(x)
```

```
class DecoderLayer(nn.Module):
```

```
    """
```

```
    Decoder is made of self-attn, src-attn, and feed forward (defined below)
```

```
    디코더는 self_attn(self-attention), src_attn(source-attention) 그리고 feed forward (network)로 구성됨  
    """
```

```
    def __init__(self, size, self_attn, src_attn, feed_forward, dropout):
```

```
        # EncoderLayer와 유사하나, src_attn이 하나 더 추가됨
```

```
        super(DecoderLayer, self).__init__()
```

```
        self.size = size
```

```
        self.self_attn = self_attn
```

```
        self.src_attn = src_attn
```

```
        self.feed_forward = feed_forward
```

```
        self.sublayer = clones(SublayerConnection(size, dropout), 3) # 인코더보다 하나의 sublayer가 더 많기 때문에 clones N = 3
```

```
    def forward(self, x, memory, src_mask, tgt_mask):
```

```
        "Follow Figure 1 (right) for connections."
```

```
        m = memory # 인코더에는 memory가 포함되지 않았음
```

```
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, tgt_mask))
```

```
        # self_attn(query, key, value, mask),
```

```
        # 따라 두번째 sub-layer는 q는 이전 decoder의 sub-layer에서 출력된 값을 가지지만, k와 v는 인코더로부터 값을 가져오게 됨
```

```
        x = self.sublayer[1](x, lambda x: self.src_attn(x, m, m, src_mask))
```

```
        return self.sublayer[2](x, self.feed_forward)
```


Code Review: Model Architecture

- Encoder-Decoder
- Encoder
- Decoder
- Attention ◀
 - Scaled Dot Product Attn
 - Multi Head Attn
- Position-wise FFN
- Embeddings
- Positional Encoding

```
def attention(query, key, value, mask=None, dropout=None):
    """
    Compute 'Scaled Dot Product Attention'

    'Scaled Dot Product Attention' 계산하기
    단, 실제 model 생성에서는 attention이 직접 사용되지는 않고, MultiHeadedAttention이 사용됨
    """

    # query.shape = [batch_size, num_heads, seq_len, d_k],
    # Transformer 논문의 경우에는 num_heads = 8, d_model = 512일 때, d_k = 64
    d_k = query.size(-1)

    # query.shape = [batch_size, num_heads, seq_len_q, d_k]
    # key.shape = [batch_size, num_heads, seq_len_k, d_k]
    # key.transpose(-2, -1) = [batch_size, num_heads, d_k, seq_len_k]
    # matmul은 행렬의 마지막 두 차원의 값이 맞아야 한다.
    # [seq_len_q, d_k] @ [seq_len_k, d_k] -> X
    # [seq_len_q, d_k] @ [d_k, seq_len_k] -> O
    scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9) # 가려진 위치의 attention 확률 ≈ 0

    p_attn = scores.softmax(dim=-1) # Normalized Attention Score, 즉 value에 곱해지게 될 (확률 형태의) 가중치

    if dropout is not None:
        p_attn = dropout(p_attn)

    return torch.matmul(p_attn, value), p_attn # Attention output과 Attention Weight(Probability) 두 값을 반환
```

Code Review: Model Architecture

- Encoder-Decoder
- Encoder
- Decoder
- Attention ◀
 - Scaled Dot Product Attn
 - Multi Head Attn
- Position-wise FFN
- Embeddings
- Positional Encoding

```
class MultiHeadedAttention(nn.Module):
    def __init__(self, h, d_model, dropout=0.1):
        """
        Take in model size and number of heads.

        model의 크기(d_model)과 head의 갯수(h)를 입력받아 attention을 head마다 독립적으로 수행한다.
        """
        super(MultiHeadedAttention, self).__init__()
        assert d_model % h == 0

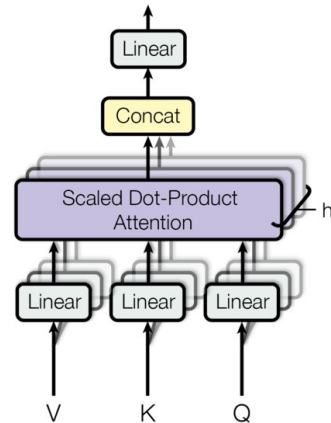
        # d_v가 항상 d_k(query와 key vector의 크기)와 같다고 가정
        self.d_k = d_model // h
        self.h = h
        self.linears = clones(nn.Linear(d_model, d_model), 4) # Q, K, V, Output Weight (Linear Projection)
        self.attn = None # Attention Score 저장
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        """Implements Figure 2(Multi-Head Attention 이미지)"""
        if mask is not None:
            # Same mask applied to all h heads.
            # [batch_size, 1, seq_len_q, seq_len_k] -> [batch_size, h, seq_len_q, seq_len_k]; broadcasting
            mask = mask.unsqueeze(1)
            nbatches = query.size(0) # = batch_size

            # 1) Do all the linear projections in batch from d_model => h x d_k
            query, key, value = [
                # linear projection -> d_model을 h와 d_k로 분할 -> seq_len과 h (위치/차원) 변경
                lin(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2) # view(-1)은 PyTorch가 설정하도록 위임, seq_len과 같은 값 입력됨
                for lin, x in zip(self.linears, (query, key, value)) # e.g., linears[0] -> Q
            ]

            # 2) Apply attention on all the projected vectors in batch.
            x, self.attn = attention(
                query, key, value, mask=mask, dropout=self.dropout
            )

            # 3) "Concat" using a view and apply a final linear.
            x = (
                x.transpose(1, 2)
                .contiguous()
                .view(nbatches, -1, self.h * self.d_k) # self.h * self.d_k = self.d_model => 원래 shape로 되돌림; concatenation 적용
            )
            del query
            del key
            del value
            return self.linears[-1](x) # Output에 대응하는 Linear Projection 거쳐 반환
```



Code Review: Model Architecture

- Encoder-Decoder
- Encoder
- Decoder
- Attention
- Position-wise FFN ◀
- Embeddings ◀
- Positional Encoding

```
class PositionwiseFeedForward(nn.Module):  
    "Implements FFN equation."  
  
    def __init__(self, d_model, d_ff, dropout=0.1):  
        super(PositionwiseFeedForward, self).__init__()  
        self.w_1 = nn.Linear(d_model, d_ff) # 512 -> 2048  
        self.w_2 = nn.Linear(d_ff, d_model) # 2048 -> 512  
        self.dropout = nn.Dropout(dropout)  
  
    def forward(self, x):  
        return self.w_2(self.dropout(self.w_1(x).relu()))
```

```
class Embeddings(nn.Module):  
    def __init__(self, d_model, vocab):  
        super(Embeddings, self).__init__()  
        self.lut = nn.Embedding(vocab, d_model)  
        self.d_model = d_model  
  
    def forward(self, x):  
        return self.lut(x) * math.sqrt(self.d_model)
```

Code Review: Model Architecture

- Encoder-Decoder
- Encoder
- Decoder
- Attention
- Position-wise FFN
- Embeddings
- Positional Encoding ◀
 - Sinusoidal Positional Encoding

```
class PositionalEncoding(nn.Module):
    "Implement the PE function."

    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * -(math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[:, : x.size(1)].requires_grad_(False)
        return self.dropout(x)
```

Code Review: Model Training

- Batches & Masking ◀
- Training Loop
- Optimizer
- Regularization
- Greedy Decoding

```
class Batch:
    """Object for holding a batch of data with mask during training."""

    def __init__(self, src, tgt=None, pad=2): # 2 = <blank>
        self.src = src
        self.src_mask = (src != pad).unsqueeze(-2)
        if tgt is not None:
            self.tgt = tgt[:, :-1]
            self.tgt_y = tgt[:, 1:]
            self.tgt_mask = self.make_std_mask(self.tgt, pad)
            self.ntokens = (self.tgt_y != pad).data.sum()

    @staticmethod
    def make_std_mask(tgt, pad):
        """Create a mask to hide padding and future words."""
        tgt_mask = (tgt != pad).unsqueeze(-2)
        tgt_mask = tgt_mask & subsequent_mask(tgt.size(-1)).type_as(
            tgt_mask.data
        )
        return tgt_mask
```

Code Review: Model Training

- Batches & Masking
- Training Loop ◀
- Optimizer
- Regularization
- Greedy Decoding

```
for i, batch in enumerate(data_iter):
    out = model.forward(
        batch.src, batch.tgt, batch.src_mask, batch.tgt_mask
    )
    loss, loss_node = loss_compute(out, batch.tgt_y, batch.ntokens)
    # loss_node = loss_node / accum_iter
    if mode == "train" or mode == "train+log":
        loss_node.backward()
        train_state.step += 1
        train_state.samples += batch.src.shape[0]
        train_state.tokens += batch.ntokens
        if i % accum_iter == 0:
            optimizer.step()
            optimizer.zero_grad(set_to_none=True)
            n_accum += 1
            train_state.accum_step += 1
            scheduler.step()

    total_loss += loss
    total_tokens += batch.ntokens
    tokens += batch.ntokens
    if i % 40 == 1 and (mode == "train" or mode == "train+log"):
        lr = optimizer.param_groups[0]["lr"]
        elapsed = time.time() - start
        print(
            "Epoch Step: %6d | Accumulation Step: %3d | Loss: %6.2f "
            + "| Tokens / Sec: %7.1f | Learning Rate: %6.1e"
            % (i, n_accum, loss / batch.ntokens, tokens / elapsed, lr)
        )
        start = time.time()
        tokens = 0
    del loss
    del loss_node
return total_loss / total_tokens, train_state
```

과제에서는
accum_iter = 10

- 1. 모델 forward
- 2. Loss 계산
- 3. train 모드면 backward 수행
- 4. optimizer.step() 실행
- 5. scheduler.step() 실행
- 6. loss/token 통계 누적
- 7. 로그 출력

Code Review: Model Training

- Batches & Masking
- Training Loop
- Optimizer◀
 - Warmup Learning Rate
- Regularization
- Greedy Decoding

```
def rate(step, model_size, factor, warmup):  
    """  
    we have to default the step to 1 for LambdaLR function  
    to avoid zero raising to negative power.  
    """  
    if step == 0:  
        step = 1  
    return factor * (  
        model_size ** (-0.5) * min(step ** (-0.5), step * warmup ** (-1.5))  
    )
```

Code Review: Model Training

- Batches & Masking
- Training Loop
- Optimizer
- Regularization ◀
 - Label Smoothing
- Greedy Decoding

```
class LabelSmoothing(nn.Module):
    "Implement label smoothing."

    def __init__(self, size, padding_idx, smoothing=0.0):
        super(LabelSmoothing, self).__init__()
        self.criterion = nn.KLDivLoss(reduction="sum")
        self.padding_idx = padding_idx
        self.confidence = 1.0 - smoothing
        self.smoothing = smoothing
        self.size = size
        self.true_dist = None

    def forward(self, x, target):
        assert x.size(1) == self.size
        true_dist = x.data.clone()
        true_dist.fill_(self.smoothing / (self.size - 2))
        true_dist.scatter_(1, target.data.unsqueeze(1), self.confidence)
        true_dist[:, self.padding_idx] = 0
        mask = torch.nonzero(target.data == self.padding_idx)
        if mask.dim() > 0:
            true_dist.index_fill_(0, mask.squeeze(), 0.0)
        self.true_dist = true_dist
        return self.criterion(x, true_dist.clone().detach())
```


Code Review: Model Training

- Batches & Masking
- Training Loop
- Optimizer
- Regularization
- Greedy Decoding ◀

```
def greedy_decode(model, src, src_mask, max_len, start_symbol):
    memory = model.encode(src, src_mask)
    ys = torch.zeros(1, 1).fill_(start_symbol).type_as(src.data)
    for i in range(max_len - 1):
        out = model.decode(
            memory, src_mask, ys, subsequent_mask(ys.size(1)).type_as(src.data)
        )
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.data[0]
        ys = torch.cat(
            [ys, torch.zeros(1, 1).type_as(src.data).fill_(next_word)], dim=1
        )
    return ys
```

Machine Translation

- Task Description

- Model: Transformer
- Dataset: multi30k from WMT 2016
- Goal: Over 0.15 BLEU-4 Score

- Train/Test Process

- Train
- Test

⇒ Code from Annotated Transformer

⇒ Add & Modify Some Part of the Original Code

- Data Loader
- Training Path
- BLEU4 Scoring ...

```
def corpus_bleu(predictions, references, max_n=4):
    """
    BLEU4 Score 계산을 목적으로 하는 함수

    - predictions: 예측 Token Sequences
    - references: 정답 Token Sequences
    - max_n: 1-gram부터 n-gram까지 비교
    """
    clipped = [0] * max_n
    total = [0] * max_n
    pred_len = 0
    ref_len = 0

    for pred, ref in zip(predictions, references):
        pred_len += len(pred)
        ref_len += len(ref)
        for n in range(1, max_n + 1):
            pred_counts = ngram_counter(pred, n)
            ref_counts = ngram_counter(ref, n)
            clipped[n - 1] += sum(
                min(count, ref_counts[ngram])
                for ngram, count in pred_counts.items()
            )
            total[n - 1] += max(len(pred) - n + 1, 0)

    if pred_len == 0 or ref_len == 0:
        return 0.0

    precisions = []
    for match_count, total_count in zip(clipped, total):
        if total_count == 0:
            precisions.append(0.0)
        elif match_count == 0:
            precisions.append(1.0 / (2 * total_count))
        else:
            precisions.append(match_count / total_count)

    brevity_penalty = 1.0 if pred_len > ref_len else math.exp(1 - ref_len / pred_len)
    return brevity_penalty * math.exp(
        sum(math.log(max(precision, 1e-12)) for precision in precisions) / max_n
    )
```

Machine Translation

• Results

• Arguments

epochs = 8 | batch-size = 32 | eval-batch-size = 32
lr = 1.0 | warmup = 3000 | accum-iter = 10
grad-clip = 1.0 | max-len = 72 | min-freq = 2
label-smoothing = 0.1

• BLEU4 Score

test_bleu4=0.2541

Best valid BLEU-4: 0.2392 | test_loss=1.9894 | test_acc=0.5893 | test_bleu4=0.2541

Example 0 =====

Source Text (Input) : Ein Mann mit einem orangefarbenen Hut , der etwas <unk> .

Target Text (Ground Truth) : A man in an orange hat starring at something .

Model Output : A man in a orange hat is looking at something .

Example 1 =====

Source Text (Input) : Ein Boston Terrier läuft über <unk> Gras vor einem weißen Zaun .

Target Text (Ground Truth) : A Boston Terrier is running on lush green grass in front of a white fence .

Model Output : A brown and white dog is running through the grass .

Example 2 =====

Source Text (Input) : Ein Mädchen in einem Karateanzug bricht ein Brett mit einem Tritt .

Target Text (Ground Truth) : A girl in karate uniform breaking a stick with a front kick .

Model Output : A girl in a blue outfit is taking a toy .

